

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS E INFORMÁTICA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE LABORATORIO

(formato estudiante)

INFORMACIÓN BÁSICA					
ASIGNATURA:	COMPUTACIÓN GRAFICA, VISIÓN COMPUTACIONAL Y MULTIMEDIA (E)				
TÍTULO DE LA PRÁCTICA:	CONTROLADORES DE ENTRADA DE PRIMERA Y TERCERA PERSONA				
NÚMERO DE PRÁCTICA:	04	AÑO LECTIVO:	5to	NRO. SEMESTRE:	NOVENO
FECHA DE PRESENTACIÓN	dd/mm/aaaa	HORA DE PRESENTACIÓN	hh/mm/ss		
INTEGRANTE (s) Sencia Ale Bryan Daniel				NOTA (0-20)	
DOCENTE(s): MBA Mg. Ing. RENE ALONSO NIETO VALENCIA.					

RESULTADOS Y PRUEBAS
Link del video: VistadeCamaras SISTEMA DE CÁMARAS Y CONTROLADOR FPS/TPS a. Arquitectura del Sistema de Control Para este laboratorio se implementó un sistema híbrido que permite la navegación fluida en un entorno tridimensional, utilizando componentes avanzados de Unity y el paquete Cinemachine. i. Componente Character Controller Se optó por el uso de un Character Controller en lugar de un Rigidbody para gestionar las colisiones físicas. Esto permite un movimiento más preciso y evita que el personaje "patine" o se caiga de forma irreal en pendientes, facilitando el control directo sobre el vector de movimiento. ii. El Rol de la Main Camera y Cinemachine Brain La Main Camera actúa como el "cerebro" del sistema. Se le añadió el componente Cinemachine Brain, el cual no renderiza una vista fija, sino que actúa como un director de orquesta que monitorea todas las cámaras virtuales de la escena y realiza transiciones suaves (blends) entre ellas según su prioridad.

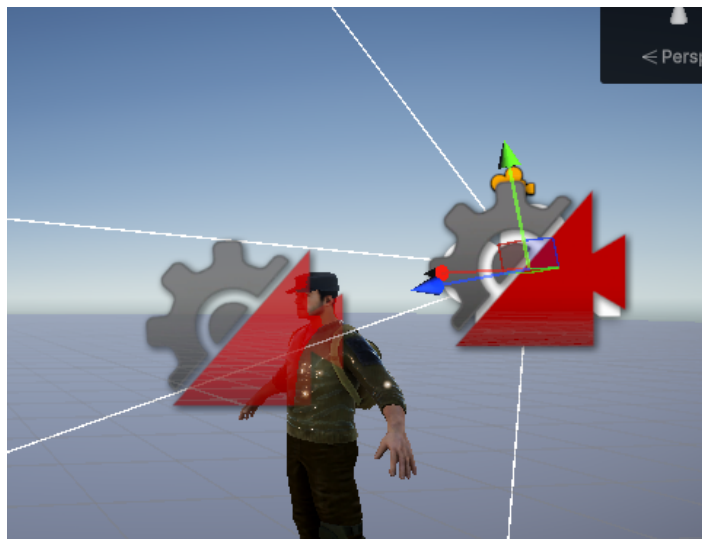
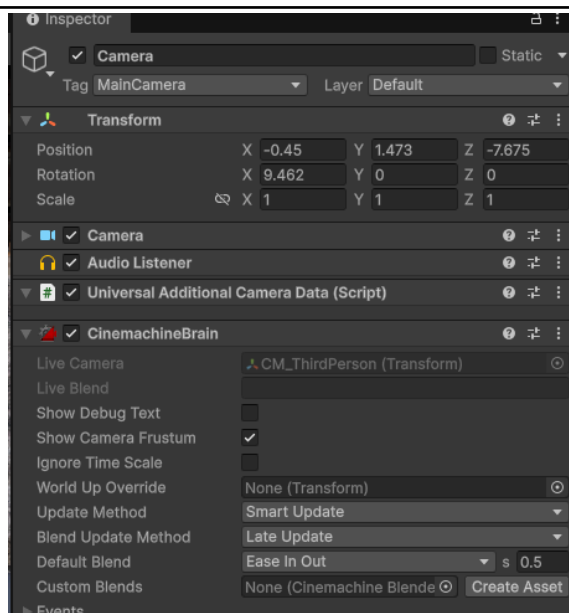


Figura 1: Configuración de la Main Camera con el componente Cinemachine Brain encargado de la interpolación entre estados de cámara.

b. Configuración de Cámaras Virtuales

Se crearon dos cámaras virtuales para ofrecer distintas experiencias de juego sin necesidad de múltiples cámaras físicas.

i. Cámara de Primera Persona (CM_FirstPerson)

Para lograr un efecto de "vista de ojos" (POV) realista, se configuró el Body como 3rd Person Follow.

1. Ajuste Técnico: Se establecieron los valores de Damping en 0 y Camera Distance en 0.
2. Justificación: Esto garantiza que la cámara esté "anclada" exactamente en los ojos del modelo sin ningún retraso de movimiento, evitando mareos en el usuario.

ii. Cámara de Tercera Persona (CM_ThirdPerson)

Se utilizó el módulo Orbital Transposer para permitir que la cámara orbite alrededor del soldado.

1. Control de Ejes: Se vinculó el eje X de la cámara al movimiento del mouse.
2. Cinemachine Collider: Se añadió como extensión para detectar obstáculos y evitar que la cámara atravesase las paredes de la penitenciaría.

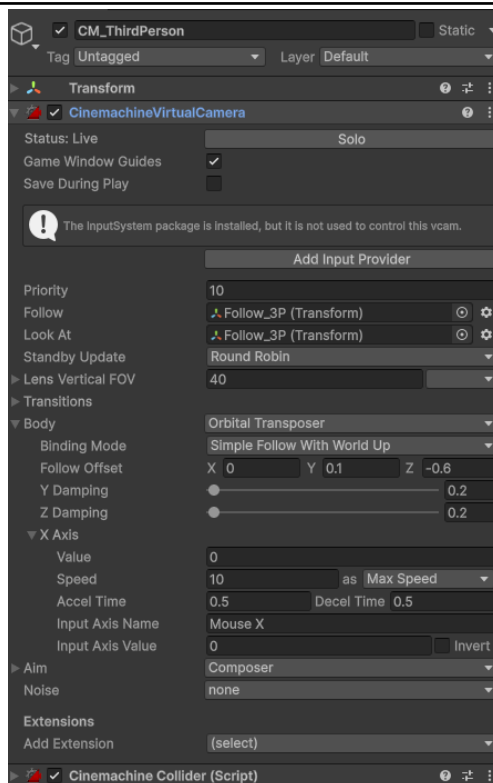


Figura 2: Vista del Inspector de la CM_ThirdPerson detallando el uso de Orbital Transposer y la gestión del eje X para la rotación con el mouse.

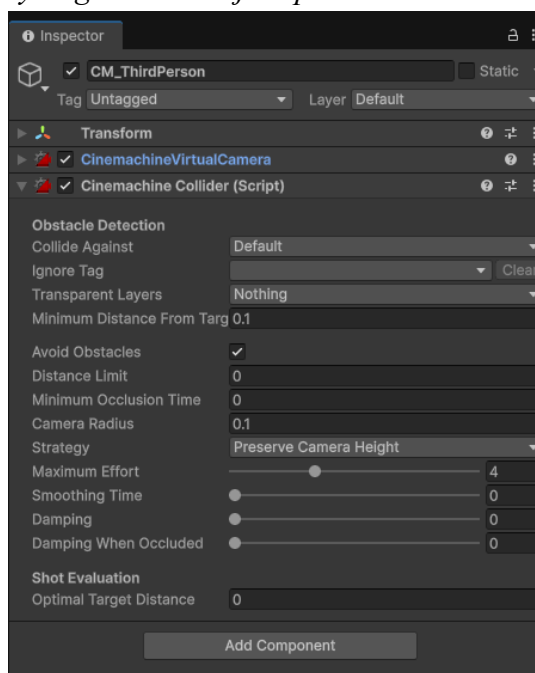


Figura 3: Implementación de la extensión de colisiones para evitar oclusiones visuales en interiores.

c. Implementación de Lógica de Programación (Scripts)

Se desarrollaron dos scripts específicos en C# para separar la lógica de movimiento de la lógica de visualización.

i. PlayerController: Movimiento y Rotación Sincronizada

Este script es el corazón del control. Se arrastró la Main Camera a la variable pública cam por una razón crítica: el personaje debe saber hacia dónde mira la cámara para orientar su propio cuerpo.

1. Lógica de Rotación: `transform.rotation = Quaternion.Euler(0, cam.eulerAngles.y, 0);` asegura que el soldado siempre dé la cara hacia donde el jugador está mirando con el mouse.
2. Movimiento Local (Strafe): Al usar `transform.forward` y `transform.right`, el personaje puede retroceder y desplazarse lateralmente de forma coherente con su modelo 3D.

```

public class PlayerController : MonoBehaviour
{
    public CharacterController controller;
    public Transform cam;
    public float speed = 2.5f;

    private float gravity = -9.81f;
    private Vector3 velocity;

    void Update()
    {
        if (controller.isGrounded && velocity.y < 0)
        {
            velocity.y = -2f;
        }

        transform.rotation = Quaternion.Euler(0, cam.eulerAngles.y, 0);

        float horizontal = Input.GetAxis("Horizontal");
        float vertical = Input.GetAxis("Vertical");

        Vector3 move = transform.right * horizontal + transform.forward * vertical;

        controller.Move(move * speed * Time.deltaTime);

        velocity.y += gravity * Time.deltaTime;
        controller.Move(velocity * Time.deltaTime);
    }
}

```

Figura 4: Lógica de programación para el desplazamiento relativo y sincronización de rotación con la cámara.

ii. SwitchCams: Gestión de Prioridades

Este script controla qué cámara domina la pantalla. Cinemachine siempre muestra la cámara con la Priority más alta. Al presionar "V", el script intercambia los valores (0 y 10), provocando que el sistema realice la transición automática.

```

public class SwitchCams : MonoBehaviour
{
    public CinemachineVirtualCamera v1P;
    public CinemachineVirtualCamera v3P;

    void Update()
    {
        if (Input.GetKeyDown(KeyCode.V))
        {
            if (v3P.Priority > v1P.Priority)
            {
                v3P.Priority = 0;
                v1P.Priority = 10;
            }
            else
            {
                v3P.Priority = 10;
                v1P.Priority = 0;
            }
        }
    }
}

```

Figura 5: Fragmento de código del PlayerController destacando la sincronización de rotación con el eje Y de la Main Camera.

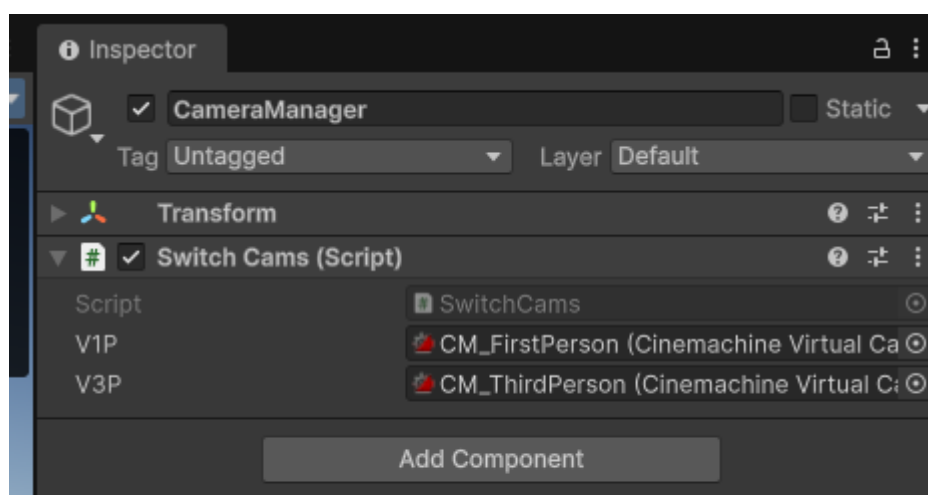


Figura 6: Vinculación de las cámaras virtuales en el Inspector para la ejecución del cambio de perspectiva.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS E INFORMÁTICA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 6

CONCLUSIONES

La integración de la herramienta Cinemachine en el flujo de trabajo del motor Unity permite gestionar sistemas de visualización complejos mediante un nivel de abstracción elevado. Esta tecnología reduce drásticamente la carga de programación manual al automatizar el cálculo de las transiciones entre las distintas perspectivas de juego, permitiendo que el sistema se encargue de la interpolación matemática entre las cámaras virtuales de forma nativa y eficiente.

La implementación de un sistema de movimiento relativo a la orientación de la cámara optimiza significativamente la experiencia de navegación del usuario en entornos tridimensionales. Al sincronizar el eje de avance del soldado con el vector de vista actual del lente, se logra una respuesta de control intuitiva que elimina la desorientación espacial y garantiza que los comandos de entrada se traduzcan en desplazamientos coherentes respecto a la posición de la cámara.

El uso del componente Cinemachine Collider resuelve de manera efectiva los errores de oclusión visual y el traspaso de mallas sólidas en escenarios con geometría cerrada. Este algoritmo de detección de obstáculos ajusta la posición de la cámara virtual en tiempo real para evitar que el plano de recorte atraviese las paredes del nivel, lo que asegura que la visibilidad del objetivo se mantenga constante sin necesidad de implementar comprobaciones de rayos personalizadas.

La arquitectura de software basada en la división de responsabilidades entre los scripts PlayerController y SwitchCams facilita el mantenimiento y la futura expansión de las funcionalidades del sistema. Mientras que el controlador del jugador gestiona los vectores de fuerza y la rotación física del modelo, el gestor de cámaras se encarga exclusivamente de la lógica de prioridades, estableciendo un flujo de trabajo modular que sigue los principios de diseño de software orientado a objetos.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS E INFORMÁTICA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 7

METODOLOGÍA DE TRABAJO

REFERENCIAS Y BIBLIOGRAFÍA

- [2] Gordon, V. S., & Clevenger, J. L. (2020). *Computer Graphics Programming in OpenGL with C++*. Stylus Publishing, LLC.
- [2] D. P. Kothari, G. Awari, D. Shrimankar , A. Bhende (2019), *Mathematics for Computer Graphics and Game Programming: A Self-Teaching Introduction*